

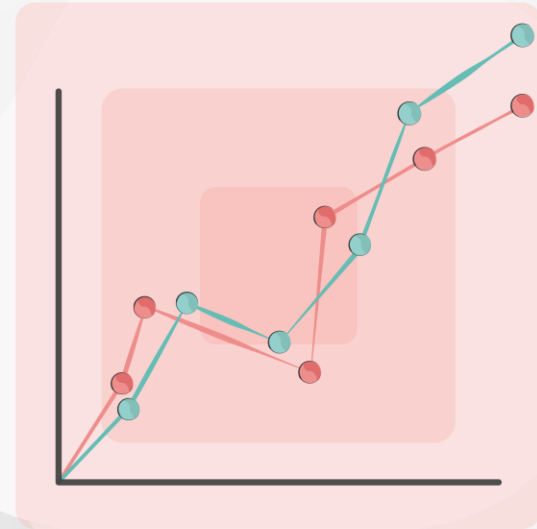


Data engineering

Module: Data pipelines

Topic: Testing strategies and debugging techniques

**Welcome to today's
webinar.**

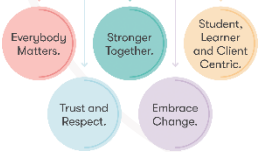


Ice breaker

Discussion...

- How are you feeling today? Motivated, happy, etc.?
- What is your key takeaway from the e-learning topic?
- What is one key skill or insight you hope to gain from today's session on testing strategies and debugging techniques?

Building Careers
Through Education



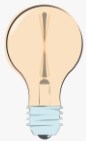
**Submit your responses to the
chat or turn on your
microphone**



The £10 Billion NHS IT Disaster

When testing and oversight fail at scale

- Largest public-sector IT project in UK history
- Aimed to modernise NHS data systems
- Poor testing, stakeholder resistance, and delays
- Dismantled after £10B spent with limited results



If you don't test with real users and real data, you risk building the wrong thing - at the highest possible cost.



NHS computer issues linked to patient harm: **Image source:** [bbc.co.uk](https://www.bbc.co.uk)

e-learning recap

Reflecting on your learning...

The e-learning for this topic, covered the following areas:

1. Why Testing Matters in Data Engineering
2. Types of Testing: Unit, Integration, System, UAT
3. Automated Testing Tools & Frameworks
4. Designing Effective Test Cases & Coverage
5. Debugging Techniques & Common Pipeline Failures



- Do you have any questions about any of these areas?
- Did everything in the e-learning make sense?

Building Careers
Through Education



Q&A discussion

Webinar Agenda

Today, we will cover the following:

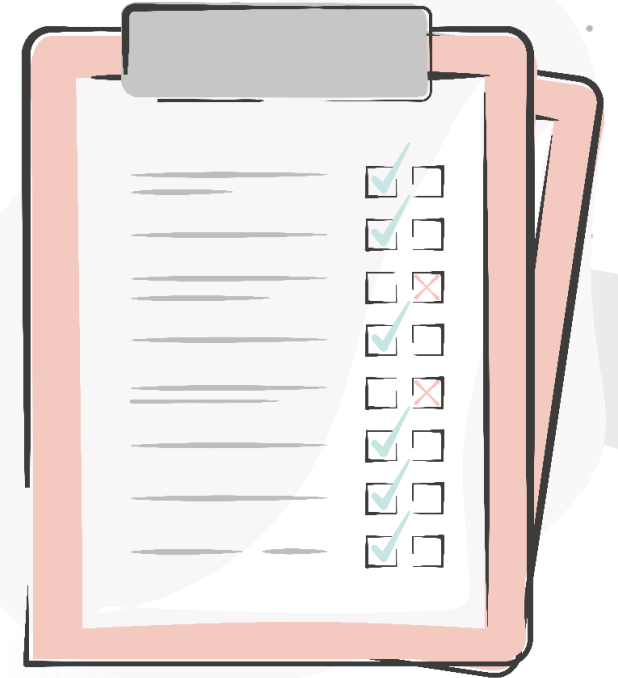
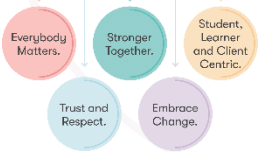
Recap

1. Why testing matters in data engineering
2. Types of testing in data pipelines
3. Automated testing frameworks
4. Designing techniques and tools
5. Common pipeline failures

Hands on challenge

1. Hackathon
2. Summary
3. Q&A

Building Careers
Through Education

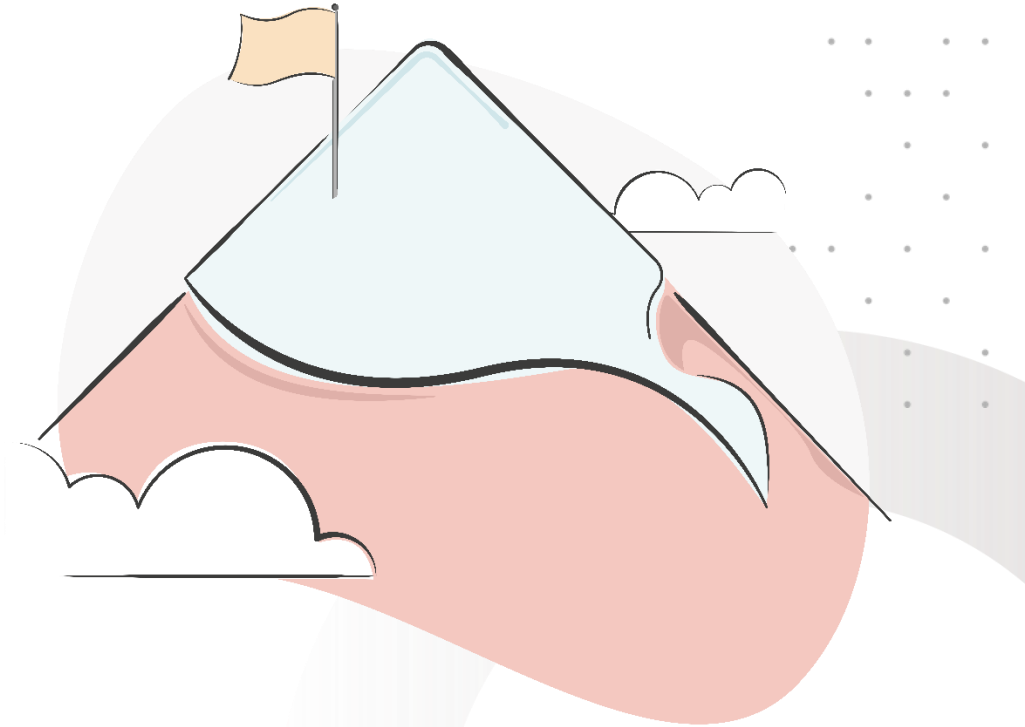


Session aim and objectives

By the end of this session, you should be able to:

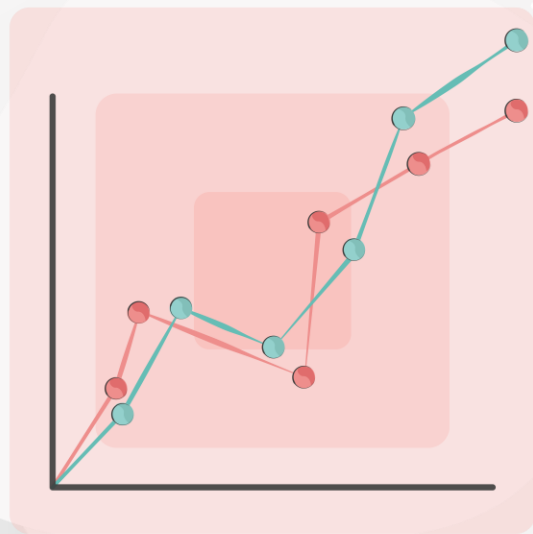
- Apply key testing strategies to validate the performance, reliability, and integrity of data pipelines.
- Use automated testing frameworks and debugging tools to identify and resolve common issues.
- Design effective test cases and debug plans that support the delivery of high-quality, maintainable data workflows.

Building Careers
Through Education





Core concept recap



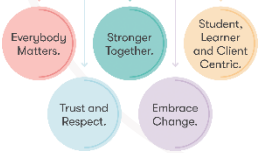
Knowledge check poll

Which of the following best describes the purpose of regression testing in a data pipeline?

- A. Verifying that each transformation produces expected outputs in isolation
- B. Checking if multiple stages of the pipeline work together end-to-end
- C. Validating that the pipeline continues to produce correct results after a change
- D. Identifying rows that contain null values or missing fields

Feedback: C – Regression testing ensures that recent changes—like new features, bug fixes, or refactors—haven't unintentionally broken existing functionality.

Building Careers
Through Education



Submit your responses to the chat!



Why testing data pipelines is critical

Small bugs, big consequences

- Data errors can break trust
- Subtle bugs delay reporting
- Testing = reliability + confidence

Q&A

Have you ever had a dashboard show the wrong numbers because of a small logic error?

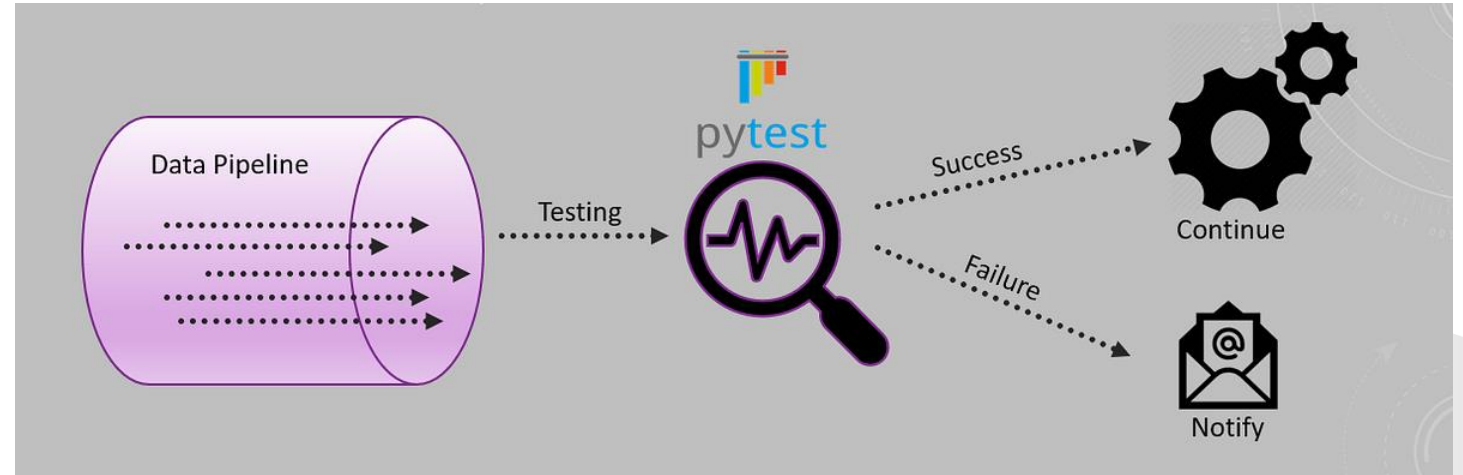


Figure: Testing isn't just about catching errors - it's about protecting trust, **image source:** devgenius.io

Types of testing

Layers of testing in data pipelines

- Unit → Integration → System → UAT
- Each layer builds confidence
- Failures often reveal earlier gaps

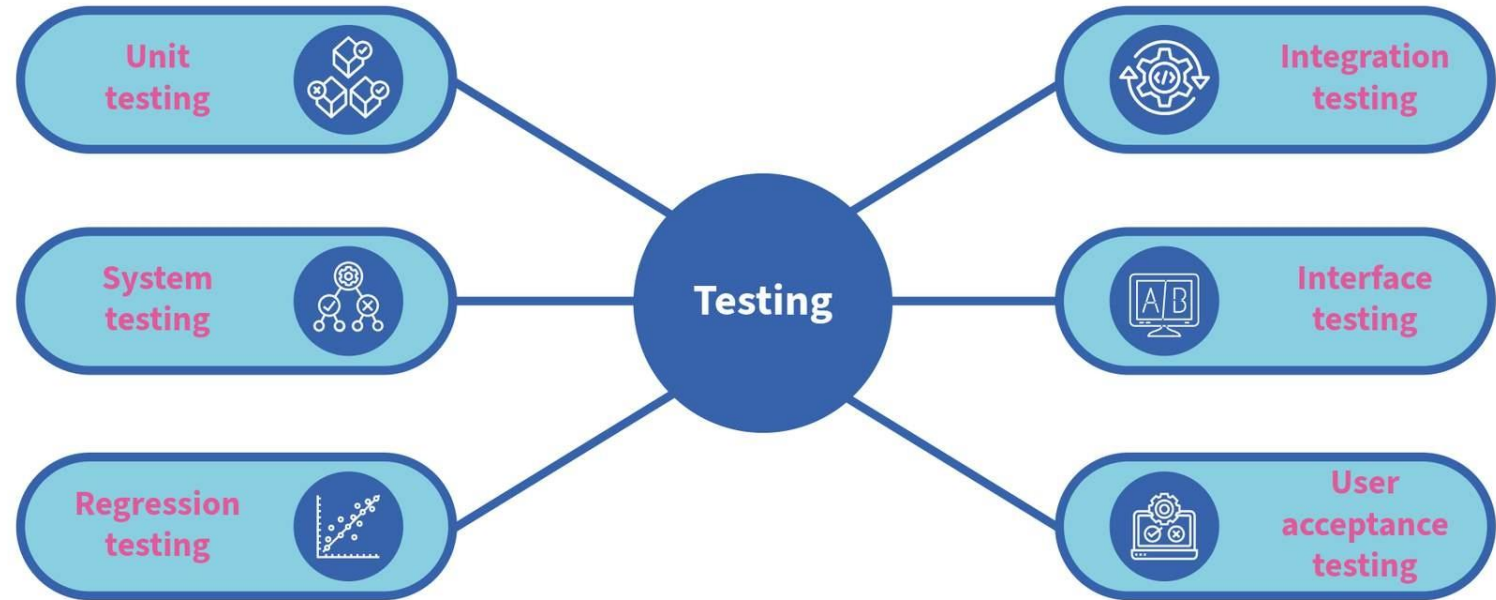


Figure: Different types of testing in data pipelines

Unit & integration testing

Isolated logic & interactions

- Unit: Test small functions with mock data
- Integration: Test modules working together
- Tools: pytest, unittest.mock

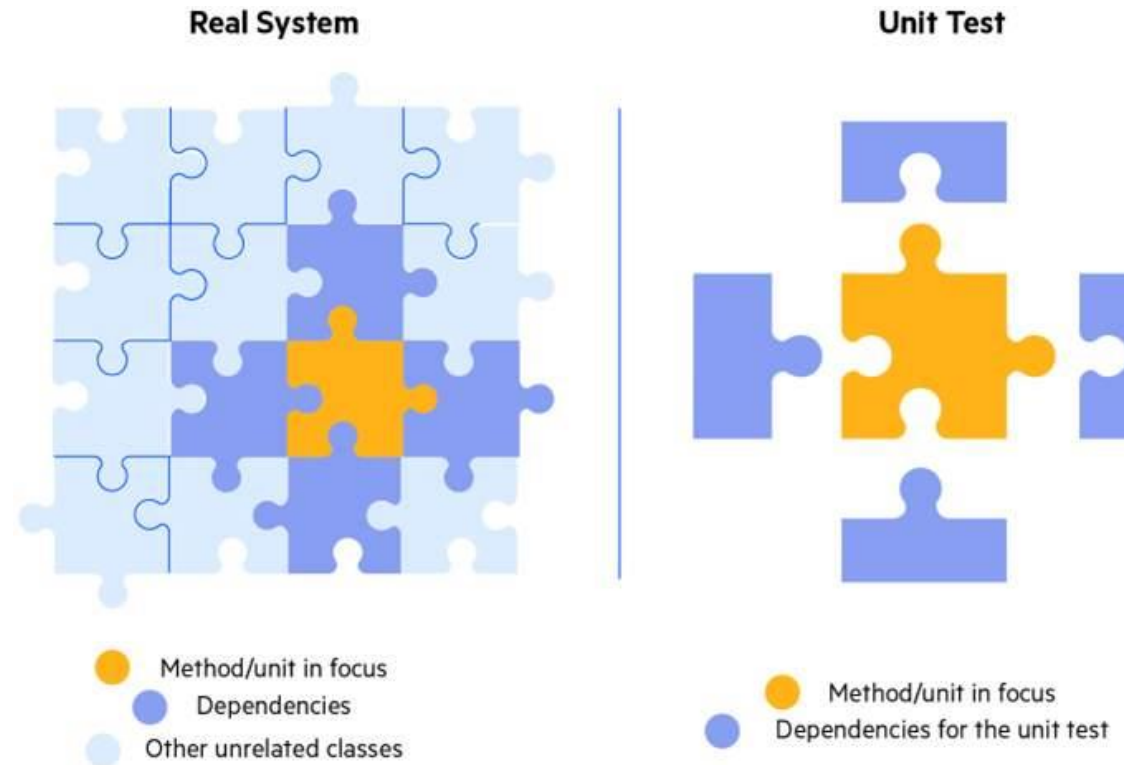


Figure: The whole system versus unit testing

System & UAT

Testing the whole pipeline

- **System:** Validate full pipeline behavior
- **UAT:** Ensure business needs are met
- Stakeholder sign-off is key

Note

Emphasise that UAT is often the last line of defence before production.



Figure: From Unit Testing to Integration Testing to finally System, or Functionality, Testing, image source: [headspin](https://www.headspin.io/)

Data Validation

Is the data right?

- Schema checks, null checks, uniqueness
- Tools: Great Expectations, Deequ
- Automate validation in pipelines

Note

Validation should be automated and run regularly.

```
1 expectation_suite_name: validate_customer_data
2 expectations:
3   - expectation_type: expect_column_values_to_not_be_null
4     kwargs:
5       column: email
6   - expectation_type: expect_column_values_to_be_between
7     kwargs:
8       column: age
9       min_value: 18
10      max_value: 99
11   - expectation_type: expect_column_values_to_be_unique
12     kwargs:
13       column: customer_id
14
```

Figure: Automated data validation ensures your pipeline's integrity by continuously checking schema, nulls, and uniqueness



Automation & CI

CI/CD for Data Pipelines

- Run tests on every commit
- Tools: GitHub Actions, Jenkins
- Prevent broken code in production

Q&A

How many of you have tests that only run on your laptop?

```
1  name: CI - Test Pipeline
2
3  on: [push]
4
5  jobs:
6    test:
7      runs-on: ubuntu-latest
8      steps:
9        - uses: actions/checkout@v3
10       - name: Run tests
11         run: |
12           pip install -r requirements.txt
13           pytest
14
```

Figure: Run your tests on every push - automated checks catch issues before they reach production.



Debugging Techniques

Debugging like a Pro

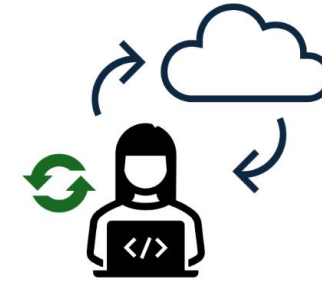
- Start with logs
- Reproduce locally
- Use breakpoint() and Git diffs

Note

Debugging is a structured process — not guesswork.

```
2024-04-16 09:00 Starting pipeline...
2024-04-16 09:00 Fetching data from source
2024-04-16 09:03 Data fetch complete
ERROR ERROR Transformation failed
ValueError: could not
convert string to float: 'N/A'
WARNING WARNING Skipping record due to
transformation error
WARNING WARNING Retrying transformation
(attempt 1 of 3)
INFO INFO Sending alert email
INFO-04-06 00:05 Transformation step completed
2024-04-16 09:06 Loading data to destination
2024-04-08 00:08 Pipeline completed
```

1 - Start with the logs



2 - Reproduce the error locally

```
def find_min(nums: [int]) -> int:
    """
    Finds the smallest integer
    value in a list
    """
    smallest = 0
    for num in nums:
        breakpoint()
        if num < smallest:
            smallest = num
    return smallest
```

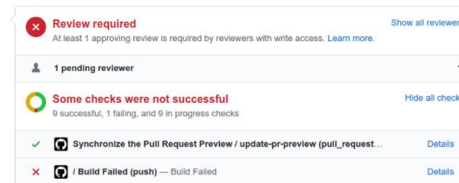
sys.breakpointhook()

pdb.set_trace()

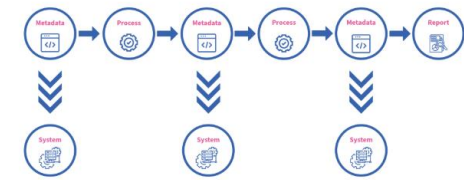
PYTHONBREAKPOINT

define custom behaviour
1. custom functions
2. remote debuggers
3. default behaviour

3 - Use breakpoints and debuggers

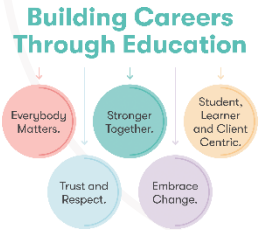


4 - Review recent changes (Version Control)



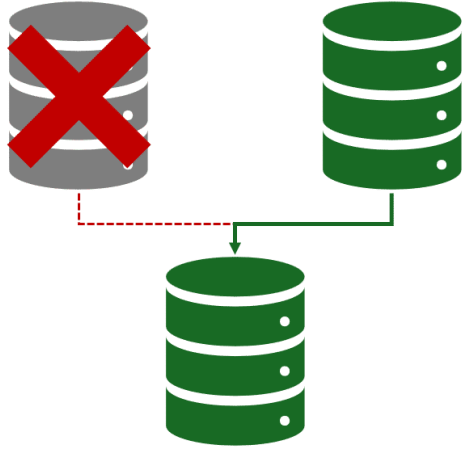
5 - Trace the data

Figure: Debugging is a structured process - start with logs, isolate locally, and trace changes with confidence.



Common pipeline failures

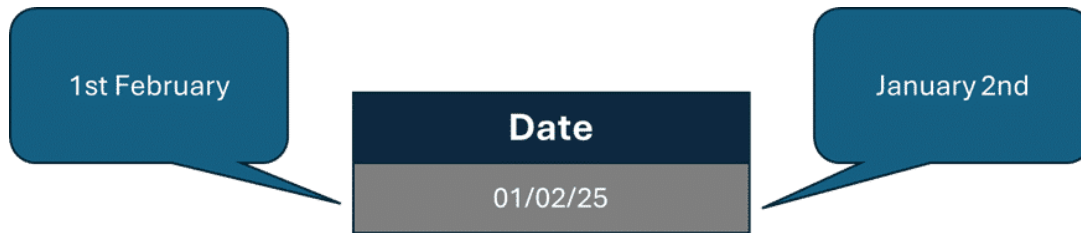
What breaks pipelines?



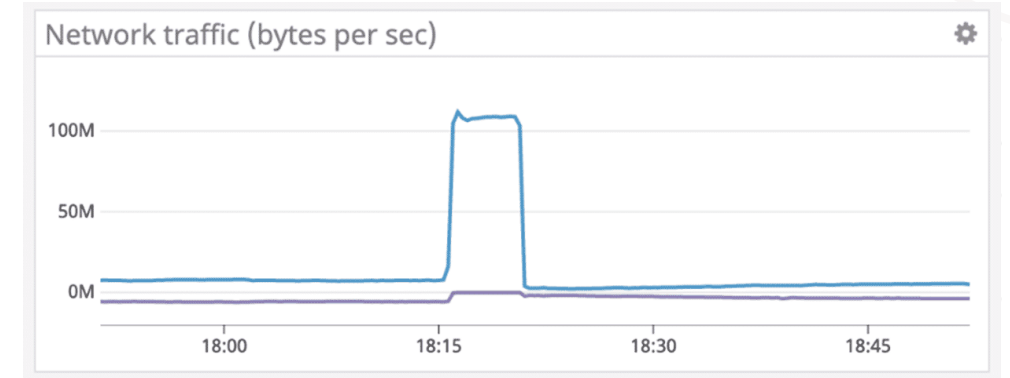
Schema Drift and Source Changes

Column
123
456
789
123

Nulls, Duplicates and Dirty Data



Logic Errors in Transformations



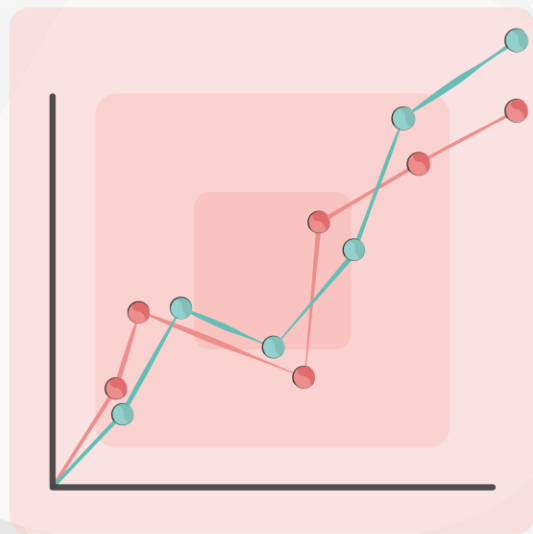
Data Volume Spikes and Performance Bottlenecks



Downstream Failures and Broken Dependencies



Practical lab



Hackathon

Part A and B...

Your step-by-step tasks are as follows:

Part A – Pipeline Design

- Design, implement, and test a data pipeline
- Merge sunrise/sunset and weather data for Edinburgh
- Generate minute-level temperature estimates for 2012

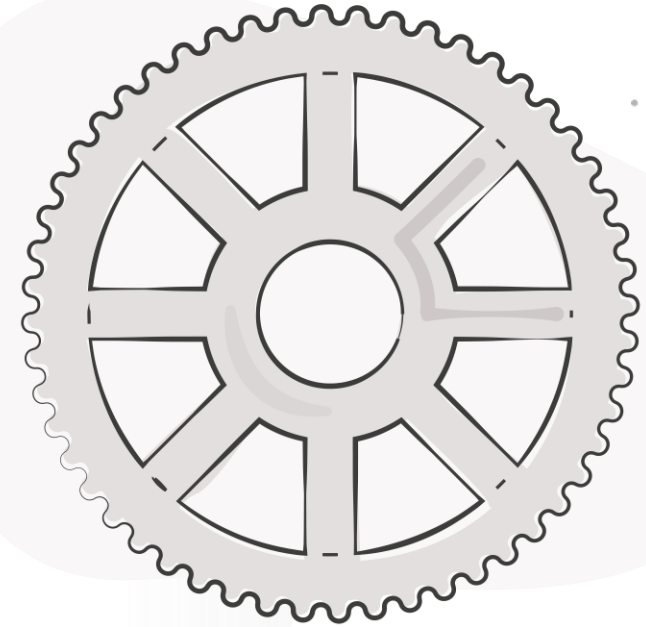
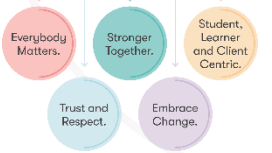
Part B – Pipeline Reuse & Adaptation

- Execute a peer's pipeline on new data (London & Hertfordshire)
- Assess documentation, adapt for schema differences
- Validate output and provide feedback

Files Provided in the Hub:

- Hackathon brief

Building Careers
Through Education



Practical challenge

Hackathon extension

Part C and D...

Part C – Big Data Model Building (Extension)

- Model relationship: temperature ↔ equipment performance
- Link model to customer complaints
- Simulate global warming impact on mast reliability
- Present findings to a non-technical audience
- Bonus: Identify peak usage times (weekday vs weekend)

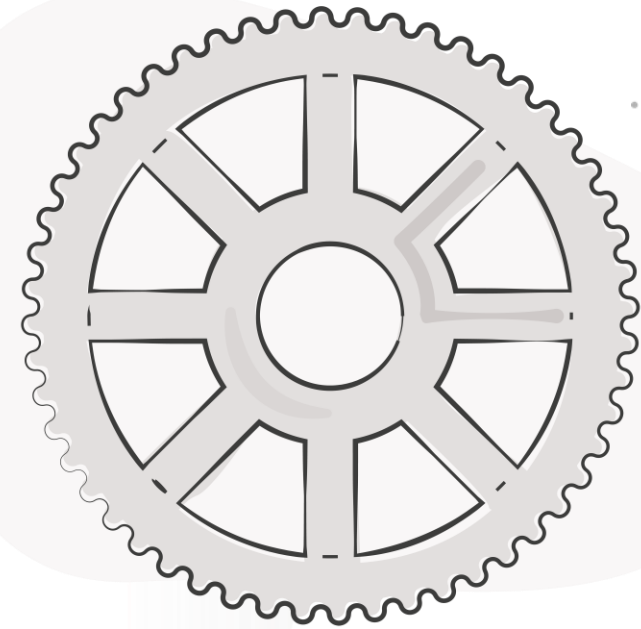
Part D – Compare Big Data Sets (Extension)

- Apply & adapt model to 8 new masts (Hertfordshire)
- Adjust for local temperature differences
- Identify unusual mast behavior
- Evaluate significance of anomalies
- Present conclusions clearly to a lay audience

Files Provided in the Hub:

- Hackathon brief

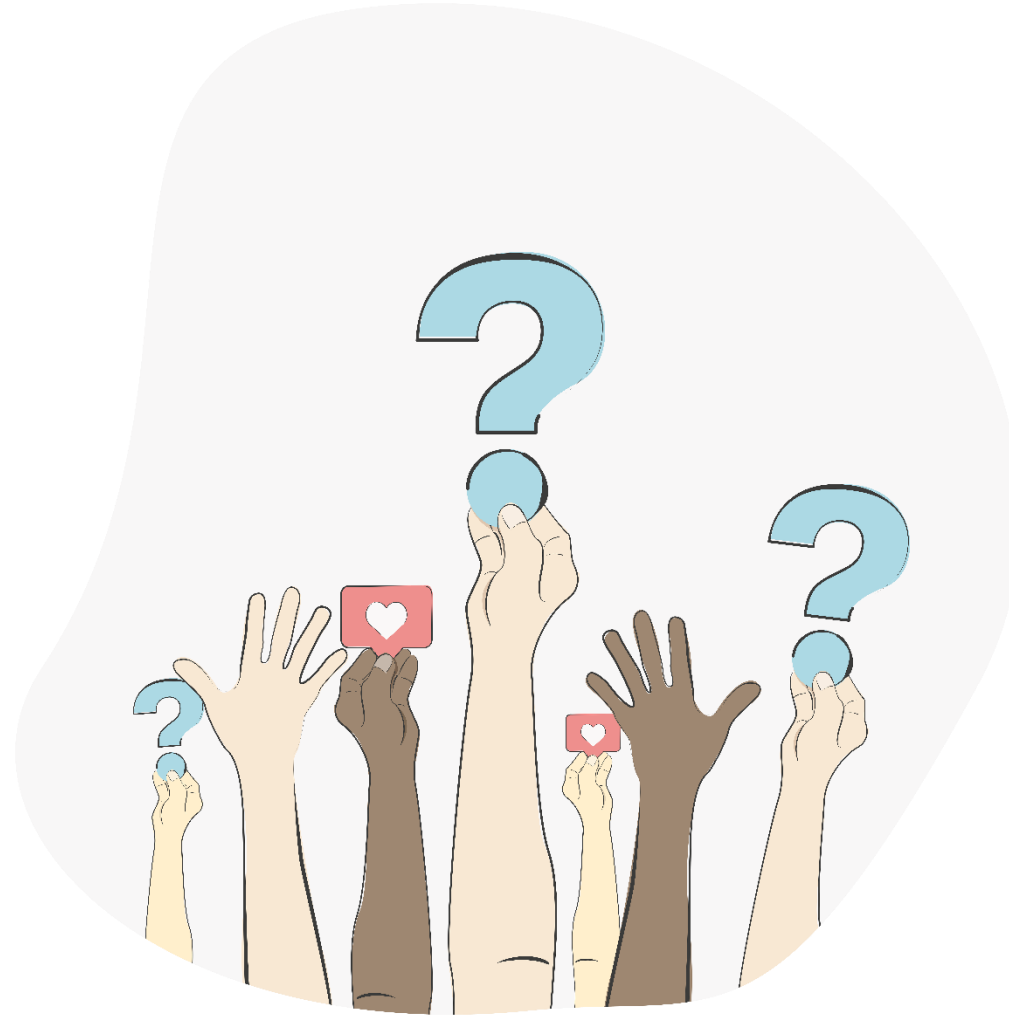
Building Careers
Through Education



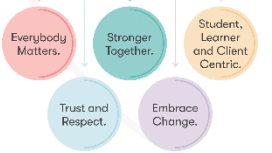
Practical challenge



Any questions or feedback?



Building Careers
Through Education





Thank you

